

React + TypeScript

Node.js 22

Docker

ECR

ECS Fargate

ALB

RDS MySQL

S3 + CloudFront

CloudWatch

# Product List on AWS End-to-End Deployment Guide

From `git clone` to a highly-available, monitored, 2-AZ production-style stack

A hands-on, picture-first walkthrough for developers who know React and Node.js but are new to AWS. Every service is explained the same way: **what it is** → **why the app needs it** → **how to configure it** → **how traffic flows** → **how to verify** → **common mistakes** → **how to troubleshoot**.

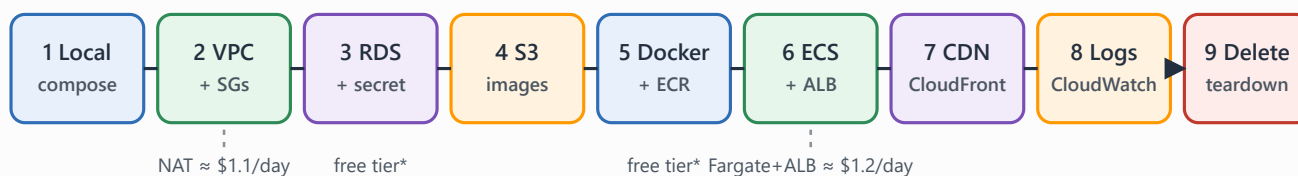
Repository: [shashikanth077/aws-node-react-deployment-github-actions](https://github.com/shashikanth077/aws-node-react-deployment-github-actions) · Deployed and verified live: eu-north-1 (Stockholm), 2 October 2026

Chapters 1–13 teach the design; **chapter 14 is the as-built record** of the real deployment: resource IDs, the verification evidence, the problems we hit and how we fixed them. Optional parts (custom domain, CloudWatch alarms/SNS, GitHub Actions) are documented but were not executed.

# ≡ Contents

|    |                                                                                                                 |
|----|-----------------------------------------------------------------------------------------------------------------|
| 1  | The big picture — what we build and how a request travels                                                       |
| 2  | The application — React, Node API, MySQL, images                                                                |
| 3  | Run it locally first                                                                                            |
| 4  | Networking — VPC, subnets, route tables, IGW, NAT, security groups                                              |
| 5  | RDS MySQL and Secrets Manager — database without hard-coded passwords                                           |
| 6  | S3 — product images and the React build                                                                         |
| 7  | Docker image and ECR                                                                                            |
| 8  | ECS Fargate — cluster, task definition, service, 2 tasks                                                        |
| 9  | Application Load Balancer and target group                                                                      |
| 10 | CloudFront — HTTPS, one site for React + API + images (optional custom domain)                                  |
| 11 | CloudWatch — logs, metrics, alarms                                                                              |
| 12 | Troubleshooting map                                                                                             |
| 13 | Cost, teardown and cheat-sheet                                                                                  |
| 14 | <b>As-built: our live deployment</b> — real resources, verification evidence, problems solved, one-page runbook |

## The build order (and rough running cost)

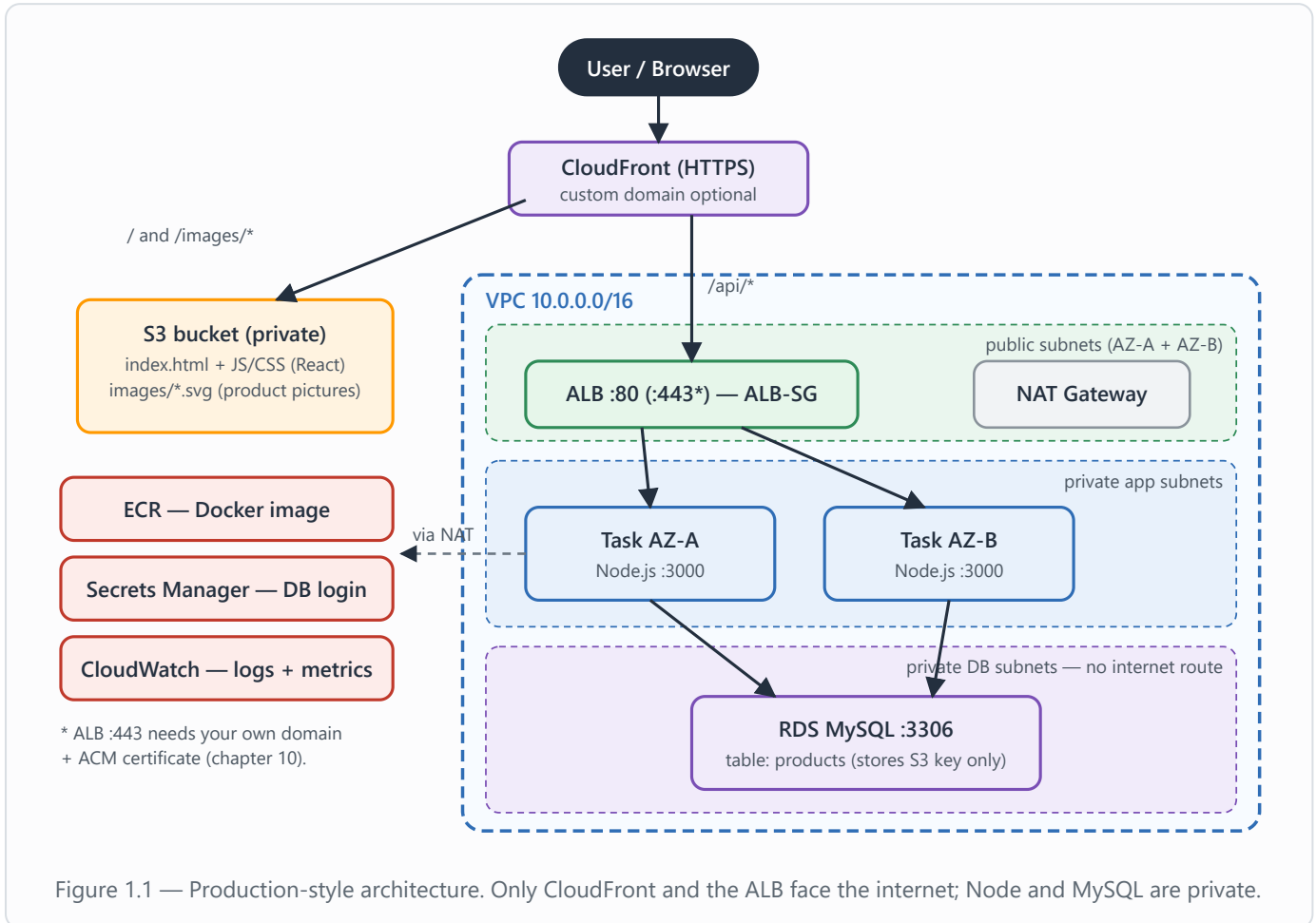


Whole stack ≈ \$2–3 per day while running — build in one or two sittings, then tear down (step 9). \*older-account Free Tier; check Billing → Free Tier.

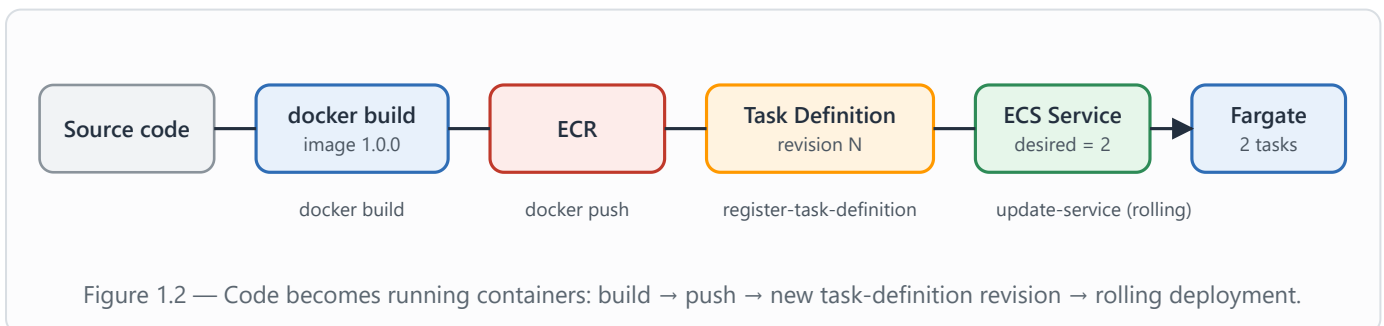
Figure 0.1 — Follow the steps in dependency order: each one needs the previous one.

# 1 The big picture

We deploy a deliberately tiny **Product List** app on a realistic architecture. The app is small so the *infrastructure* is the lesson: private networking, two Availability Zones, a load balancer, containers, a managed database, secrets, a CDN and monitoring.



## Deployment flow

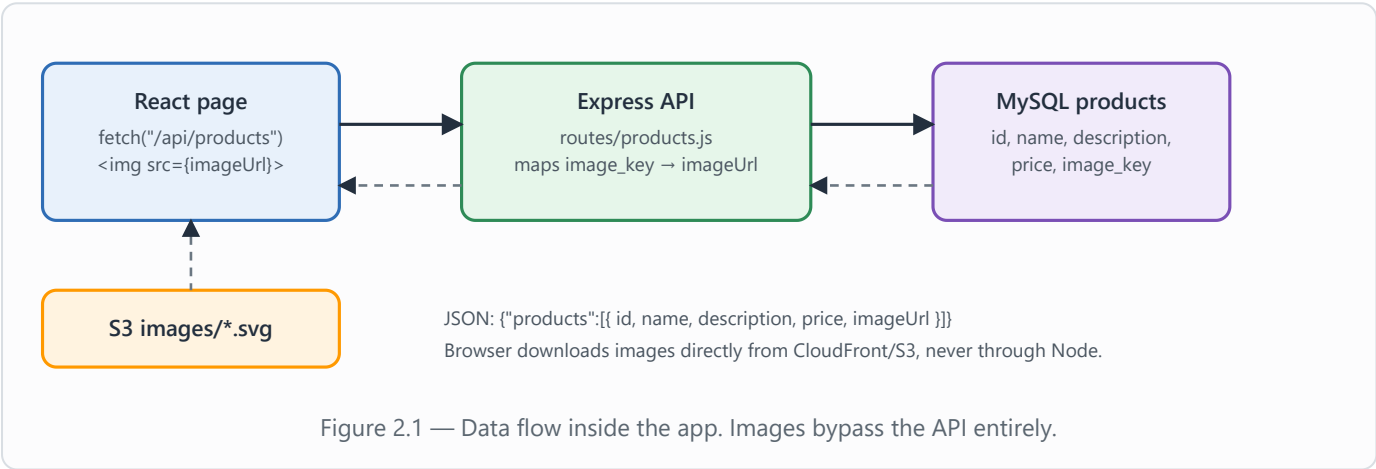


## Names used in every command

| Thing            | Name                                                                                                                                                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VPC              | <code>product-vpc</code> 10.0.0.0/16                                                                                                                                                                                                    |
| Subnets          | <code>public-a</code> 10.0.1.0/24 · <code>public-b</code> 10.0.3.0/24 · <code>private-app-a</code> 10.0.2.0/24 · <code>private-app-b</code> 10.0.4.0/24 · <code>private-db-a</code> 10.0.5.0/24 · <code>private-db-b</code> 10.0.6.0/24 |
| Security groups  | <code>alb-sg</code> · <code>ecs-sg</code> · <code>rds-sg</code>                                                                                                                                                                         |
| RDS              | <code>product-db</code> , database <code>productsdb</code> , subnet group <code>product-db-subnet-group</code>                                                                                                                          |
| S3 / ECR         | <code>product-app-&lt;ACCOUNT_ID&gt;</code> / <code>product-api</code>                                                                                                                                                                  |
| ECS              | cluster <code>product-cluster</code> · task family <code>product-api</code> · service <code>product-api-service</code>                                                                                                                  |
| Load balancer    | <code>product-alb</code> · target group <code>product-api-tg</code>                                                                                                                                                                     |
| IAM roles / logs | <code>productApiTaskExecutionRole</code> · <code>productApiTaskRole</code> / <code>/ecs/product-api</code>                                                                                                                              |

## 2 The application

| Layer    | Tech                         | Key points                                                                                                                                                                                             |
|----------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend | React 19 + TypeScript + Vite | Fetches <code>/api/products</code> (same origin → no CORS). Loading, error + retry states. Card: image, name, price, description.                                                                      |
| Backend  | Node 22 + Express            | GET <code>/api/products</code> , GET <code>/health</code> (liveness for ALB), GET <code>/health/ready</code> (checks DB). Env-var config, JSON logs, central error handler, graceful SIGTERM shutdown. |
| Database | MySQL 8 (RDS)                | <code>products</code> table, 5 seed rows, connection pool ( <code>mysql2</code> , size 10/task, TLS with RDS CA bundle).                                                                               |
| Images   | S3 (private)                 | DB stores only the <b>key</b> ( <code>images/lamp.svg</code> ); API builds the URL from <code>IMAGE_BASE_URL</code> .                                                                                  |



### Why `/health` does not touch the database

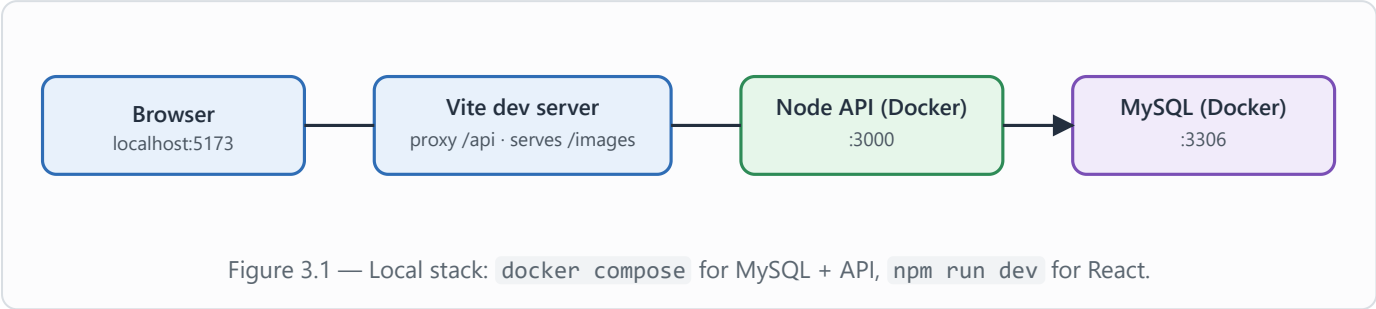
If the ALB health check queried MySQL and the DB hiccuped, the ALB would mark *both* tasks unhealthy and take the whole site down. Liveness ( `/health` ) answers "is Node alive?"; readiness ( `/health/ready` ) answers "can it reach the DB?" and is for humans and debugging.

### Repository layout

```
backend/  Dockerfile · db/schema.sql + seed.sql · src/{config,logger,db,app,server}.js · routes/ · scrip
ts/init-db.js
frontend/ Vite + React + TS (src/App.tsx, api.ts, components/ProductCard.tsx)
assets/   images/*.*.svg → uploaded to S3
infra/    task-definition.json · IAM policies · ecr-lifecycle-policy.json · GitHub Actions example
scripts/  render-task-def.sh · render-task-role-policy.sh
docs/     the markdown guide (this PDF is its illustrated summary)
```

### 3 Run it locally first

Never debug AWS and your code at the same time. Prove the app works on your laptop; afterwards the only new variable is AWS.



```
docker compose up -d --build
docker compose run --rm api npm run db:init           # → {"msg":"db_seeded"}
curl -s localhost:3000/health                         # {"status":"ok"}
curl -s localhost:3000/api/products | head -c 300
cd frontend && npm install && npm run dev              # http://localhost:5173
```

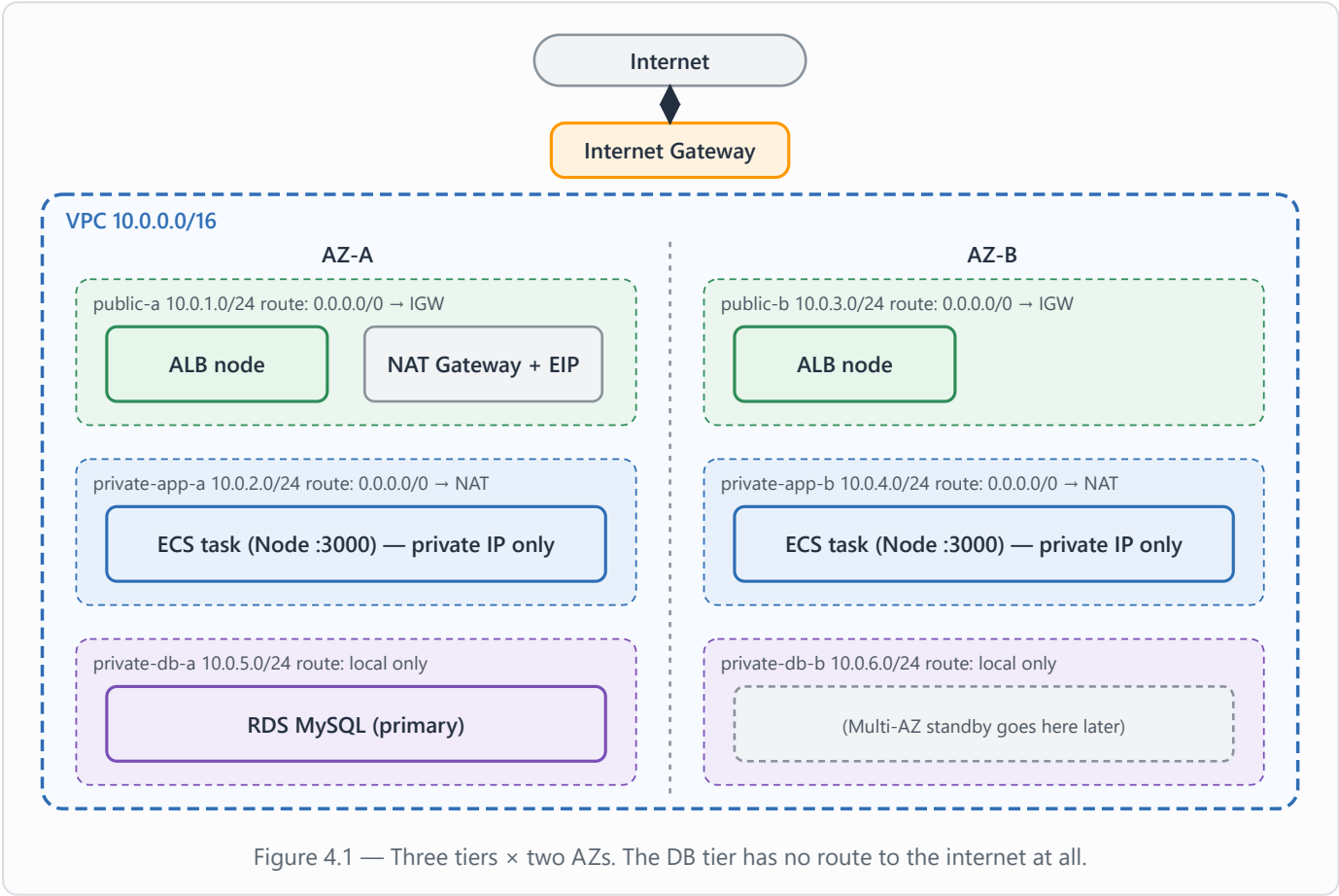
| Variable              | Local               | In AWS                             |
|-----------------------|---------------------|------------------------------------|
| DB_HOST / DB_NAME     | db / productsdb     | RDS endpoint / productsdb          |
| DB_USER + DB_PASSWORD | root + dev password | not set                            |
| DB_SECRET_ARN         | —                   | Secrets Manager ARN                |
| DB_SSL                | false               | true (verified with RDS CA bundle) |
| IMAGE_BASE_URL        | empty               | empty (same-origin via CloudFront) |

**TIP** Stop the API and reload the page: you see the error state with a Try-again button — the same behaviour you will see if anything in AWS breaks.

# 4 Networking

## Mental model in plain words

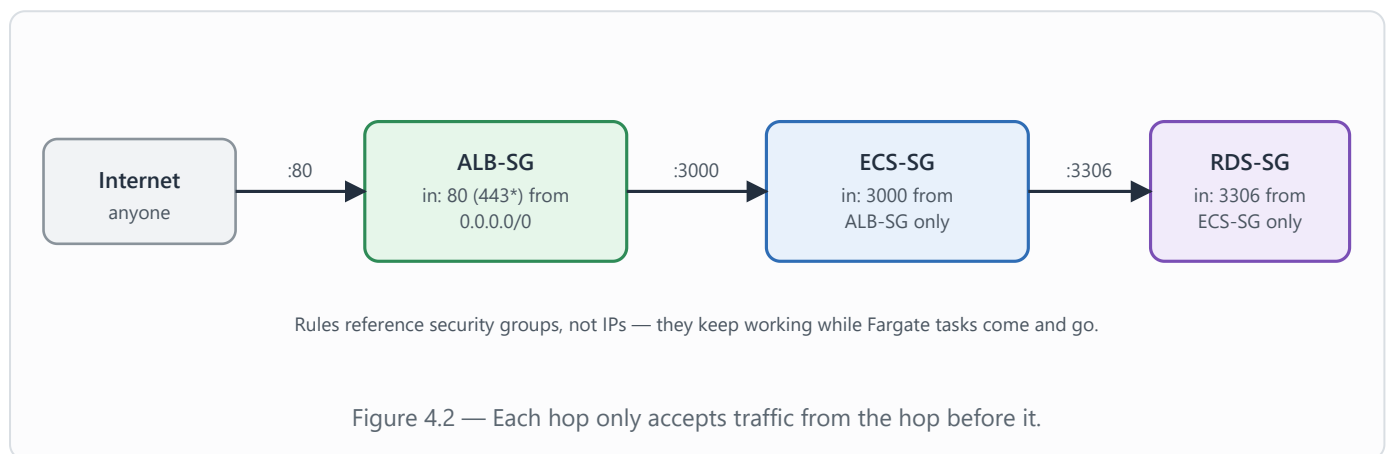
| Piece                  | Think of it as                                                                                                                                                      |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VPC                    | Your own private data centre inside AWS (10.0.0.0/16).                                                                                                              |
| Subnet                 | A room, pinned to <i>one</i> Availability Zone (a physically separate building).                                                                                    |
| Internet Gateway (IGW) | The front door to the street. No IGW → no internet in or out.                                                                                                       |
| Route table            | Signposts: "to reach X, go through Y". A subnet is <i>public</i> only because its signpost says 0.0.0.0/0 → IGW.                                                    |
| NAT Gateway            | A one-way mail slot. Private tasks can send out (pull image, fetch secret) and get replies; strangers cannot start a conversation. Lives in a <i>public</i> subnet. |
| Security group         | A firewall around each resource; stateful; rules can name another security group as the source.                                                                     |



## Route tables

| Table              | Attached to        | Routes                                                                                  |
|--------------------|--------------------|-----------------------------------------------------------------------------------------|
| product-public-rt  | public-a, public-b | 10.0.0.0/16 → local · <b>0.0.0.0/0 → IGW</b>                                            |
| product-private-rt | private-app-a/b    | 10.0.0.0/16 → local · <b>0.0.0.0/0 → NAT</b> · S3 prefix list → gateway endpoint (free) |
| product-db-rt      | private-db-a/b     | 10.0.0.0/16 → local <i>only</i>                                                         |

## Security groups — the chain



**WHY NOT EXPOSE 3000 OR 3306?** Port 3306 open to the world is how databases get scanned, brute-forced and ransomed within minutes — only Node needs MySQL, so only Node's SG may connect. Port 3000 open would let people bypass the load balancer (no health routing, no central logging/TLS/WAF); the tasks have no public IP anyway. One controlled front door: the ALB.

### Create it (console)

VPC → **Create VPC** → **"VPC and more"**: name `product`, 10.0.0.0/16, 2 AZs, 2 public + 2 private subnets with the CIDRs above, **NAT gateways: In 1 AZ**, VPC endpoint **S3 Gateway**, DNS hostnames on. Then add subnets `private-db-a/b` (10.0.5.0/24, 10.0.6.0/24) with a route table that has only the local route. Create `alb-sg` → `ecs-sg` → `rds-sg` in that order.

```
export VPC_ID=$(aws ec2 describe-vpcs --filters Name=tag:Name,Values=product-vpc --query 'Vpcs[0].VpcId' -
-output text)
aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID \
  --query 'Subnets[].[Tags[?Key==`Name`]][0].Value,AvailabilityZone,CidrBlock]' --output table
```

**PITFALLS** NAT placed in a private subnet · no `0.0.0.0/0 → NAT` on the private table (tasks time out pulling images) · CIDR ranges instead of SG references · opening 3306 "temporarily" · DB subnets in one AZ · wrong console region.

**VERIFY** VPC → Resource map: public subnets → IGW route; private-app → NAT route; DB subnets → no 0.0.0.0/0; NAT is "Available" and in a *public* subnet. Stuck? **Reachability Analyzer** names the exact blocking rule.

## 5 RDS MySQL + Secrets Manager

**RDS** is a managed MySQL: AWS patches, backs up and replaces hardware. **Why we need it:** containers are disposable; state must live outside them so both tasks read the same rows.

| Setting            | Value                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------|
| Engine / template  | MySQL 8.x (our build got 8.4.9) · Free tier · single DB instance                        |
| Identifier / class | product-db · db.t3.micro (or t4g.micro) · 20 GiB, no autoscaling                        |
| Credentials        | <b>Managed in AWS Secrets Manager</b> (master user admin)                               |
| Network            | product-vpc · subnet group on the two DB subnets · <b>Public access: No</b> · SG rds-sg |
| Initial database   | productsdb                                                                              |
| Extras             | Backups 1 day · Performance Insights and Enhanced Monitoring off                        |

```
aws rds describe-db-instances --db-instance-identifier product-db \
  --query 'DBInstances[0].[DBInstanceStatus,Endpoint.Address,PubliclyAccessible,MultiAZ,MasterUserSecret.SecretArn]' --output text
export DB_HOST=... export DB_SECRET_ARN=... # from the output above
```

### How Node gets the password — two IAM roles

| Role                                                      | Used by                                | For                                    | Permissions                                                         |
|-----------------------------------------------------------|----------------------------------------|----------------------------------------|---------------------------------------------------------------------|
| <b>Task execution role</b><br>productApiTaskExecutionRole | The ECS agent, before your code starts | Pull image from ECR, write log streams | AWS-managed<br>AmazonECSTaskExecutionRolePolicy                     |
| <b>Task role</b><br>productApiTaskRole                    | Your Node code, while running          | Any AWS API the app calls              | Inline: secretsmanager:GetSecretValue on <i>this one secret ARN</i> |

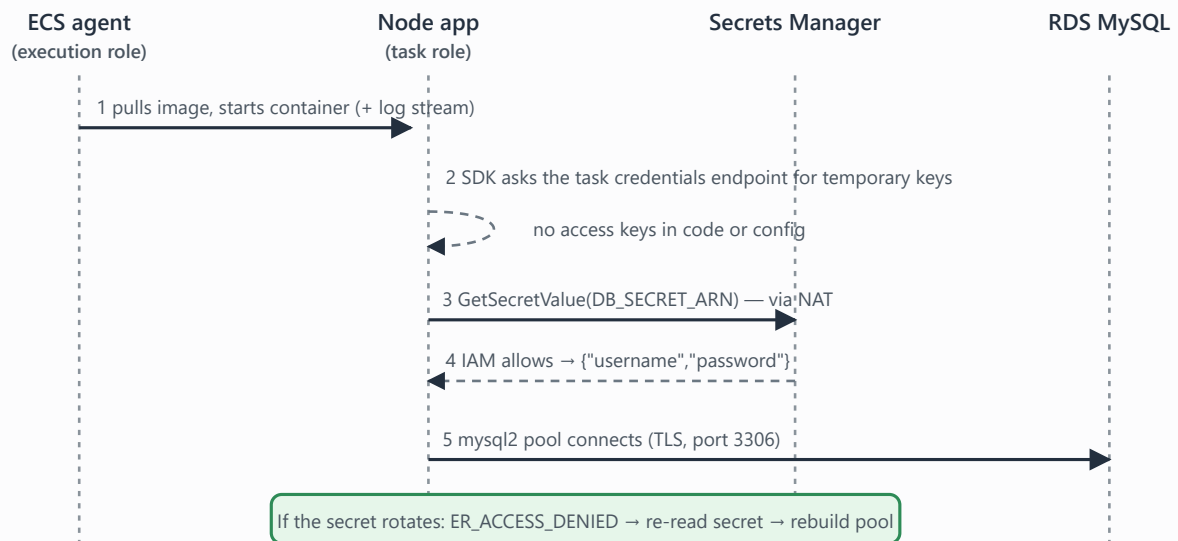


Figure 5.1 — Credentials flow. The password never appears in Git, the image, the task definition or the logs.

#### WHY NOT INJECT IT AS AN ECS "SECRET" ENV VAR? You can ( "secrets":

[{"name":"DB\_PASSWORD","valueFrom":"<arn>:password:":"}], needs the *execution* role to read it). But it is read once at task start, and RDS-managed secrets rotate every 7 days — running tasks would keep a stale password. Reading it in code with the task role self-heals and teaches the pattern you will reuse for S3/SQS.

```

aws iam create-role --role-name productApiTaskExecutionRole --assume-role-policy-document file://infra/ecs-tasks-trust-policy.json
aws iam attach-role-policy --role-name productApiTaskExecutionRole --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
aws iam create-role --role-name productApiTaskRole --assume-role-policy-document file://infra/ecs-tasks-trust-policy.json
bash scripts/render-task-role-policy.sh > infra/task-role-policy.rendered.json
aws iam put-role-policy --role-name productApiTaskRole --policy-name read-db-secret --policy-document file://infra/task-role-policy.rendered.json
  
```

**PITFALLS** Public access = Yes · forgetting the initial DB name (Unknown database) · secretsmanager:\* on \* · password in environment · no NAT/endpoint, so Secrets Manager is unreachable · fixing the wrong role (error from Node → *task* role; task fails to start → *execution* role).

## 6 S3 — images and the React build

**What:** object storage; a file is addressed by a *key* such as `images/headphones.svg`. **Why:** images do not belong in MySQL (bloat, slow) or in the Docker image (every picture change would need a redeploy). The row stores the key; S3 stores the bytes; browsers fetch them via CloudFront so Node never serves an image.



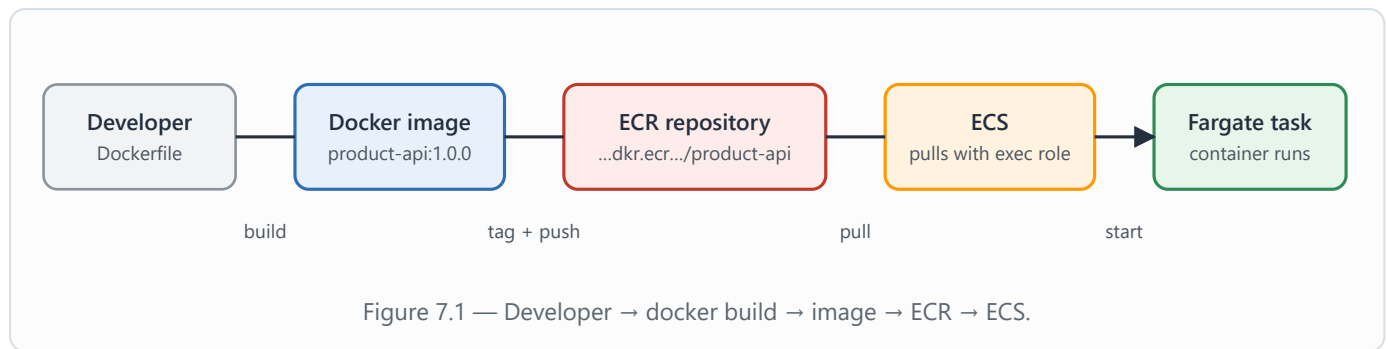
One private bucket: images/... (pictures) and / (index.html + hashed JS/CSS)

Figure 6.1 — Key in the database, bytes in S3, URL assembled at request time.

```
aws s3api create-bucket --bucket "$BUCKET" --region "$AWS_REGION" --create-bucket-configuration LocationCo
nstraint="$AWS_REGION"
aws s3api put-public-access-block --bucket "$BUCKET" --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3 sync assets/images "s3://$BUCKET/images" --cache-control "public, max-age=86400"
aws s3 presign "s3://$BUCKET/images/headphones.svg" --expires-in 300 # opens the picture; plain URL →
AccessDenied (correct!)
```

**PITFALLS** turning off Block Public Access "to make it work" · key mismatch between DB and S3 (case-sensitive, no leading "/") · very long cache on files you later overwrite · S3 returns 403 (not 404) for missing keys when you lack ListBucket.

## 7 Docker image and ECR



### What the Dockerfile does

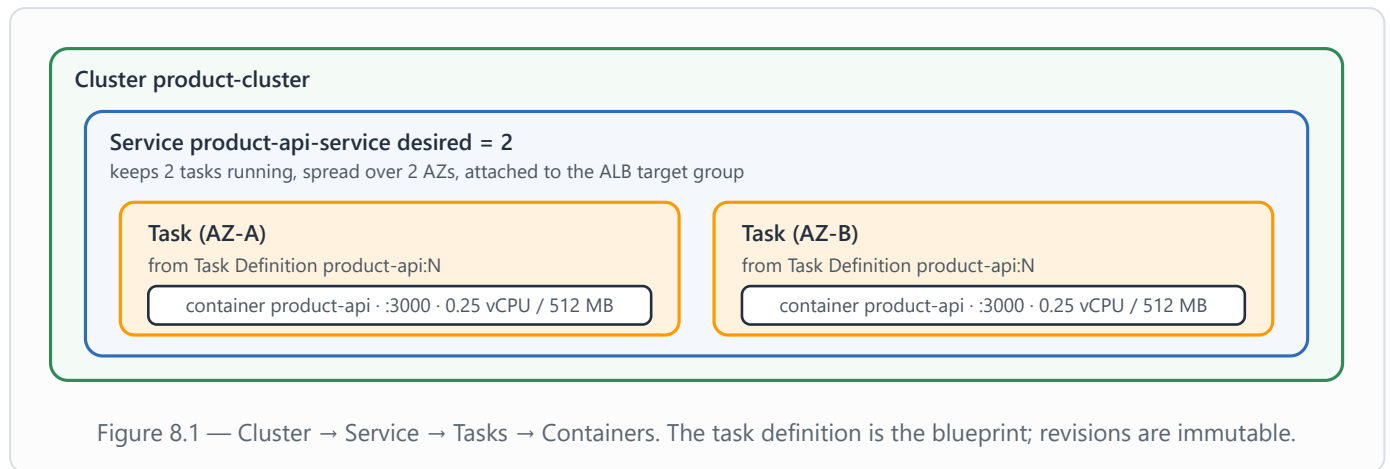
| Step                                                                  | Purpose                                                                                            |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Stage 1: <code>node:22-alpine</code> , <code>npm ci --omit=dev</code> | Exact, locked, production-only dependencies; layer cached until <code>package*.json</code> changes |
| Stage 2: fresh <code>node:22-alpine</code>                            | Small runtime image — build tooling not shipped                                                    |
| ADD <code>...global-bundle.pem</code>                                 | Amazon RDS CA bundle so the DB connection is encrypted <i>and verified</i>                         |
| <code>chown + USER node</code>                                        | Never run as root in the container                                                                 |
| CMD <code>["node", "src/server.js"]</code>                            | Exec form → Node is PID 1 and receives SIGTERM → graceful shutdown on deploys                      |
| <code>.dockerignore</code>                                            | Keeps <code>node_modules</code> , <code>.env</code> out of the image                               |

```
# 1 build  2 test locally  3 create repo  4 tag  5 authenticate  6 push
cd backend && docker build -t product-api:1.0.0 .
docker run --rm -p 3000:3000 --network aws-node-react-deployment-github-actions_default \
  -e DB_HOST=db -e DB_NAME=productsdb -e DB_USER=root -e DB_PASSWORD=localdevpassword product-api:1.0.0
aws ecr create-repository --repository-name product-api --image-scanning-configuration scanOnPush=true
aws ecr put-lifecycle-policy --repository-name product-api --lifecycle-policy-text file://infra/ecr-lifecycle-policy.json
export ECR_URI=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/product-api
docker tag product-api:1.0.0 $ECR_URI:1.0.0
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
docker push $ECR_URI:1.0.0
aws ecr describe-images --repository-name product-api --query 'imageDetails[][imageTags[0],imagePushedAt]' --output table
```

**PITFALLS** no basic auth credentials (login expires after 12 h / wrong region) · pushing `latest` and expecting ECS to notice · image built on Apple Silicon ( `exec format error` — build with `--platform linux/amd64` ) · tag without the registry prefix goes to Docker Hub.

## 8 ECS Fargate

**Fargate** = "run this container with 0.25 vCPU and 512 MB"; AWS finds the compute. No servers to patch. Each task gets its *own network interface* (private IP + security group) in `awsvpc` mode.



### Task definition, field by field

| Setting            | Value                                                   | Why                                                     |
|--------------------|---------------------------------------------------------|---------------------------------------------------------|
| Launch / network   | FARGATE, <code>awsvpc</code>                            | per-task ENI and security group                         |
| CPU / memory       | 256 / 512                                               | smallest valid size; raise to 512/1024 if memory > ~70% |
| Image              | <code>\$ECR_URI:1.0.0</code>                            | exact tag, never <code>latest</code>                    |
| Port               | 3000                                                    | what the target group forwards to                       |
| Environment        | <code>DB_HOST, DB_NAME, DB_SSL, DB_SECRET_ARN...</code> | non-secret config; the ARN is a <i>pointer</i>          |
| Roles              | <code>execution + task role</code>                      | see chapter 5                                           |
| healthCheck        | <code>wget .../health</code>                            | ECS replaces unhealthy tasks                            |
| stopTimeout / init | 30 s / true                                             | time to drain after SIGTERM; reap zombies               |
| logConfiguration   | <code>awslogs → /ecs/product-api</code>                 | <b>this ships console.log to CloudWatch</b>             |

```
aws logs create-log-group --log-group-name /ecs/product-api
aws logs put-retention-policy --log-group-name /ecs/product-api --retention-in-days 7
aws ecs create-cluster --cluster-name product-cluster
export IMAGE_URI=$ECR_URI:1.0.0 && bash scripts/render-task-def.sh
aws ecs register-task-definition --cli-input-json file://infra/task-definition.rendered.json

# one-off task: same image, different command → creates the table + 5 products
aws ecs run-task --cluster product-cluster --launch-type FARGATE --task-definition product-api \
  --network-configuration "awsvpcConfiguration={subnets=[$PRIV_A],securityGroups=[$ECS_SG],assignPublicIp=DISABLED}" \
  --overrides '{"containerOverrides":[{"name":"product-api","command":["node","src/scripts/init-db.js"]}]}'
aws logs tail /ecs/product-api --follow --since 5m      # expect: db_credentials_loaded ... db_seeded
```

## Create the service (2 tasks, 2 AZs) — after the ALB exists (next chapter)

```
aws ecs create-service --cluster product-cluster --service-name product-api-service \
  --task-definition product-api --desired-count 2 --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[$PRIV_A,$PRIV_B],securityGroups=[$ECS_SG],assignPublicIp=DISABLED}" \
  --load-balancers "targetGroupArn=$TG_ARN,containerName=product-api,containerPort=3000" \
  --health-check-grace-period-seconds 30 \
  --deployment-configuration "deploymentCircuitBreaker={enable=true,rollback=true},minimumHealthyPercent=100,maximumPercent=200"
```

## Zero-downtime rolling deployment

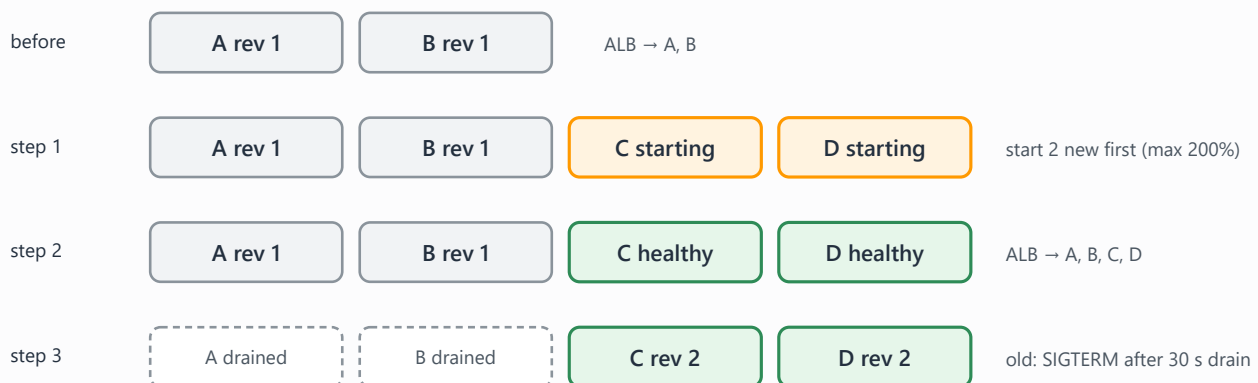
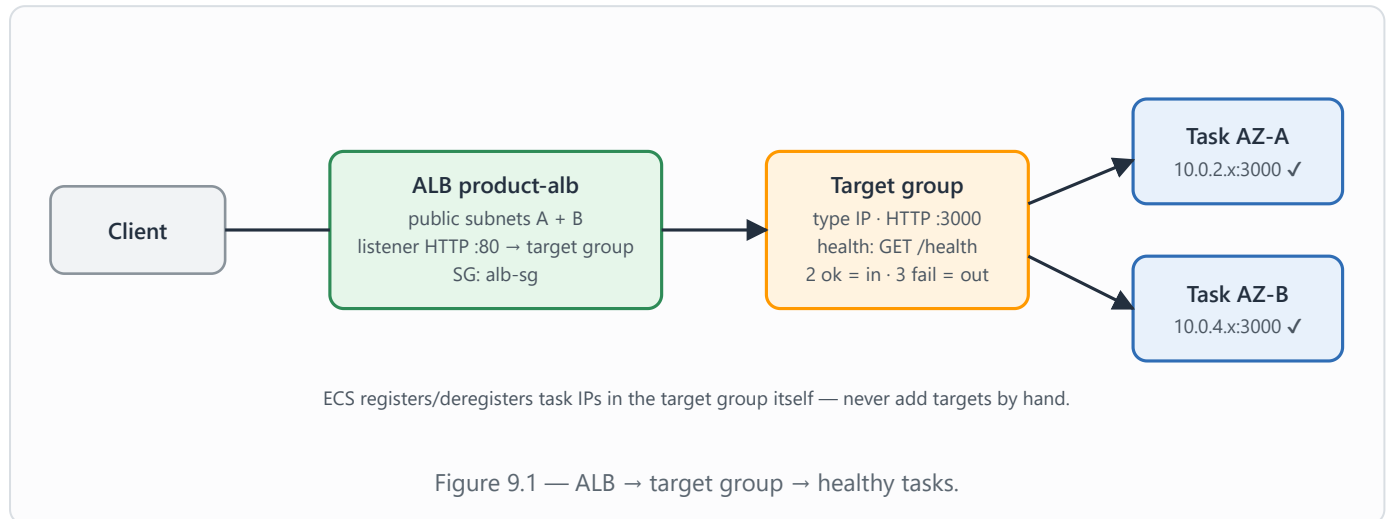


Figure 8.2 — If C or D never become healthy, the circuit breaker rolls back to revision 1 automatically.

```
# deploy a new version
docker build -t product-api:1.0.1 backend && docker tag product-api:1.0.1 $ECR_URI:1.0.1 && docker push $ECR_URI:1.0.1
export IMAGE_URI=$ECR_URI:1.0.1 && bash scripts/render-task-def.sh
aws ecs register-task-definition --cli-input-json file://infra/task-definition.rendered.json
aws ecs update-service --cluster product-cluster --service product-api-service --task-definition product-api
```

## 9 Application Load Balancer

**What:** the single front door that accepts HTTP(S), picks a healthy task and forwards the request. **Why:** task IPs change constantly and there are two; the ALB gives one stable DNS name, balances across AZs and **stops sending traffic to failing tasks**.



| Target group field | Value                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------|
| Target type        | IP (required for Fargate) · name <code>product-api-tg</code> · HTTP : 3000                      |
| Health check       | path <code>/health</code> · interval 15 s · timeout 5 s · healthy 2 · unhealthy 3 · success 200 |
| Attribute          | deregistration delay <b>30 s</b> (default 300 s makes each deploy 5 min slower)                 |

Load balancer: **Internet-facing**, IPv4, VPC `product-vpc`, subnets **public-a + public-b**, security group `alb-sg` only, listener HTTP:80 → `product-api-tg`.

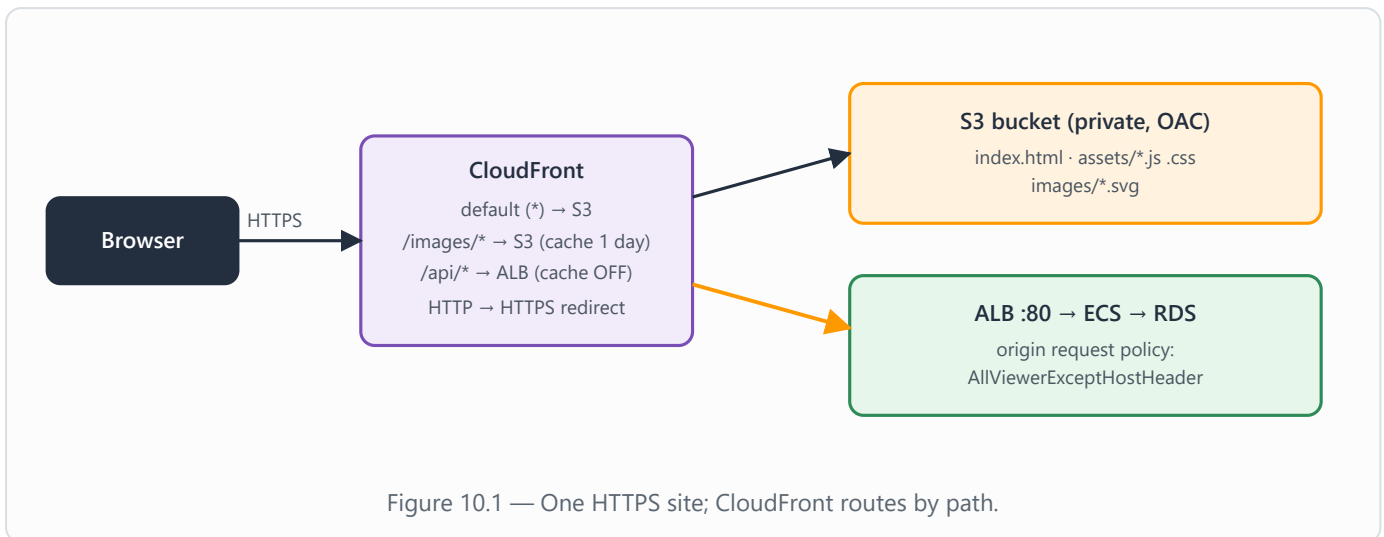
### Verify high availability

```
aws elbv2 describe-target-health --target-group-arn $TG_ARN --query 'TargetHealthDescriptions[][Target.Id,TargetHealth.State]' --output table
curl -s http://$ALB_DNS/api/products | head -c 300
TASK=$(aws ecs list-tasks --cluster product-cluster --service-name product-api-service --query 'taskArns[0]' --output text)
aws ecs stop-task --cluster product-cluster --task $TASK --reason "HA test"
for i in $(seq 1 20); do curl -s -o /dev/null -w "%{http_code}" http://$ALB_DNS/health; sleep 2; done #
200 200 200 ...
```

| ALB says                                 | Meaning                                                                             |
|------------------------------------------|-------------------------------------------------------------------------------------|
| 503                                      | no healthy targets registered → service events / stopped tasks                      |
| 502                                      | task closed the connection / crashed mid-request → app logs                         |
| 504                                      | task too slow, or SG blocks → <code>TargetResponseTime</code> , <code>ecs-sg</code> |
| <code>Target.Timeout</code>              | security group or wrong port                                                        |
| <code>Target.ResponseCodeMismatch</code> | health path wrong / not 200                                                         |

# 10 CloudFront and the React frontend

**Why CloudFront:** (1) free HTTPS on \*.cloudfront.net — an ALB :443 listener needs a domain + certificate; (2) **one origin** for everything, so no CORS and no mixed-content errors; (3) the S3 bucket stays private.



```
cd frontend && npm ci && npm run build && cd ..
aws s3 sync frontend/dist "s3://$BUCKET" --delete --exclude "images/*" --exclude "index.html" --cache-control "public, max-age=31536000, immutable"
aws s3 cp frontend/dist/index.html "s3://$BUCKET/index.html" --cache-control "no-cache"
```

Hashed JS/CSS (index-YEUI1bVh.js) can be cached for a year; index.html must always be re-checked.

| Create in the console | Settings                                                                                                                                                                                       |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Origin 1              | The bucket's regular S3 endpoint ( <b>not</b> the website endpoint) · Origin access control (new OAC) · default root object index.html · then paste the generated <b>bucket policy</b> into S3 |
| Origin 2              | ALB DNS name · HTTP only · port 80                                                                                                                                                             |
| Behavior /api/*       | origin = ALB · all HTTP methods · cache policy <b>CachingDisabled</b> · origin request policy <b>AllViewerExceptHostHeader</b>                                                                 |
| Default behavior      | origin = S3 · GET/HEAD · CachingOptimized · redirect HTTP→HTTPS · price class: North America + Europe                                                                                          |

```
curl -sI https://$CF_DOMAIN/ # 200 text/html
curl -s https://$CF_DOMAIN/api/products | head -c 250
curl -sI https://$CF_DOMAIN/images/lamp.svg # 200 image/svg+xml
curl -sI http://$BUCKET.s3.amazonaws.com/images/lamp.svg | head -n1 # 403 -> bucket is private ✓
aws cloudfront create-invalidation --distribution-id <ID> --paths "/index.html" # after each frontend deploy
```

**PITFALLS** website endpoint as origin (OAC unsupported) · no bucket policy (every request 403) · default caching on /api/\* · origin protocol HTTPS-only to an HTTP listener (502) · adding the SPA "403/404 → index.html" error rule, which would turn API 404s into HTML.

## Optional: custom domain + HTTPS on the ALB (Route 53)

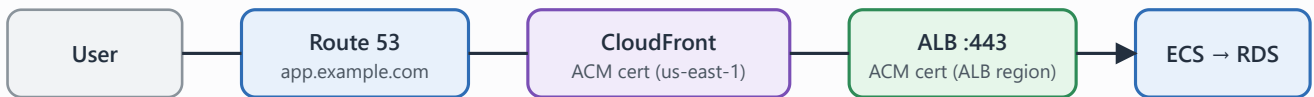
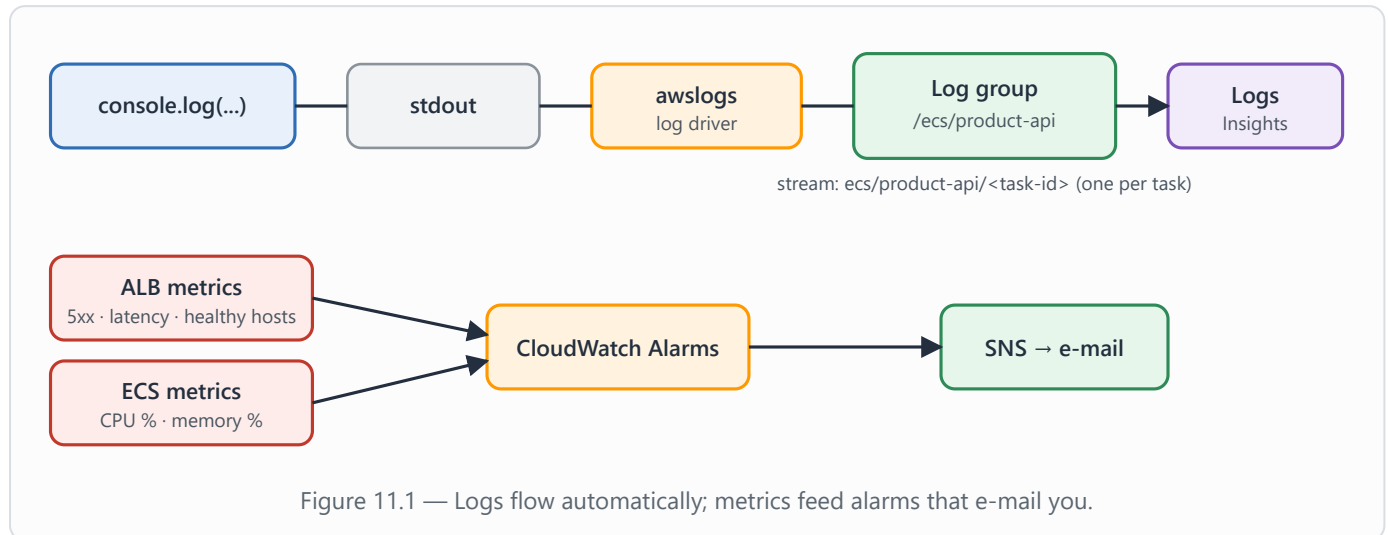


Figure 10.2 — With your own domain you get the full Route 53 → CloudFront → ALB :443 → ECS path (needs a domain, ≈ \$12/yr).

# 11 CloudWatch — logs, metrics, alarms

Containers are disposable and have no SSH; if it is not in CloudWatch, it is gone when the task dies. Our logger prints one JSON object per line with `console.log`; Docker captures stdout; the `awslogs` driver (in the task definition) forwards it. No agent, no code changes.



## Find your console.log output

Console: CloudWatch → **Logs** → **Log groups** → **/ecs/product-api** → pick a stream (two tasks = two streams) or **Search all log streams**. CLI:

```
aws logs tail /ecs/product-api --follow --since 10m
aws logs tail /ecs/product-api --since 1h --filter-pattern '"level":"error"'
# sample line:
{"time":"...", "level":"info", "msg":"request", "requestId":"Root=1-67...", "method":"GET", "path":"/api/products", "status":200, "ms":6.4}
```

## Logs Insights queries (JSON fields are discovered automatically)

```
fields @timestamp, msg, err.message, requestId | filter level = "error" | sort @timestamp desc | limit 50
filter msg = "request" | stats count() as n, avg(ms), pct(ms,95) as p95 by bin(5m)
filter msg = "request" and status >= 500 | stats count() by path
```

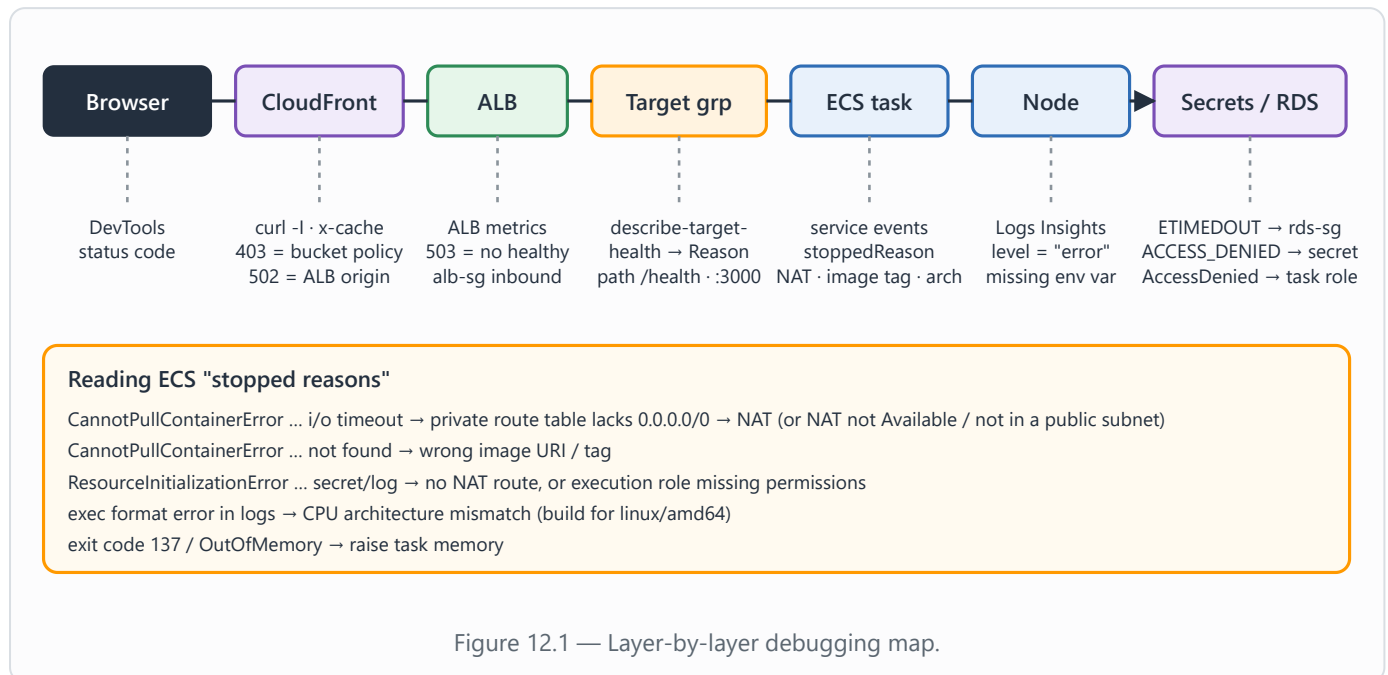
| Metric                                    | Healthy | Tells you                                    |
|-------------------------------------------|---------|----------------------------------------------|
| ALB HTTPCode_Target_5XX_Count             | 0       | your app returned 5xx                        |
| ALB HTTPCode_ELB_5XX_Count                | 0       | ALB failed: 502/503/504                      |
| ALB TargetResponseTime                    | < 0.3 s | slow app or DB                               |
| ALB HealthyHostCount / UnHealthyHostCount | 2 / 0   | tasks passing/failing health checks          |
| ECS CPUUtilization / MemoryUtilization    | < 70%   | raise size or add tasks; OOM = exit code 137 |

```
aws sns create-topic --name product-alerts
aws sns subscribe --topic-arn arn:aws:sns:$AWS_REGION:$ACCOUNT_ID:product-alerts --protocol email --notification-endpoint you@example.com
aws cloudwatch put-metric-alarm --alarm-name product-api-high-cpu --namespace AWS/ECS --metric-name CPUUtilization --statistic Average \
  --dimensions Name=ClusterName,Value=product-cluster Name=ServiceName,Value=product-api-service \
  --period 300 --evaluation-periods 2 --threshold 80 --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:$AWS_REGION:$ACCOUNT_ID:product-alerts
```

**PITFALLS** no retention (never-expiring logs cost money) · logging secrets/bodies · unstructured logs you cannot filter · alarm without a *confirmed* SNS subscription · wrong region/time range in the console.

# 12 Troubleshooting map

Follow the request from the outside in. The first layer that misbehaves is where to dig.



```
# the three commands that solve most ECS problems
aws ecs describe-services --cluster product-cluster --services product-api-service --query 'services[0].events[:6].message' --output text
aws ecs list-tasks --cluster product-cluster --desired-status STOPPED --query 'taskArns[:3]' --output text
| xargs -r aws ecs describe-tasks --cluster product-cluster --query 'tasks[].[stoppedReason,containers[0].reason]' --output text --tasks
aws logs tail /ecs/product-api --since 15m
```

| Symptom                   | Likely cause → fix                                                   |
|---------------------------|----------------------------------------------------------------------|
| Images broken, API fine   | S3 key ≠ DB image_key , or bucket policy/OAC                         |
| Old page after deploy     | Invalidate /index.html ; hard reload                                 |
| Tasks flap start/stop     | Container crash or failing health check → logs + stoppedReason       |
| Everything times out      | Run Reachability Analyzer between a task ENI and the RDS ENI on 3306 |
| Bill higher than expected | NAT / ALB / Fargate left running → Cost Explorer by service          |

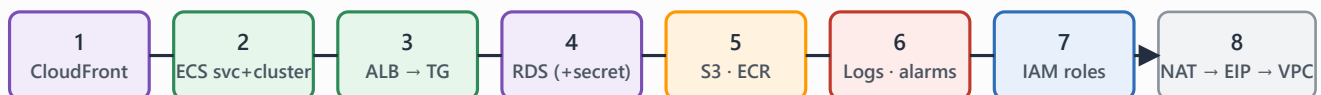
## 13 Cost, teardown and cheat-sheet

### What costs money (rough orders of magnitude)

| Service                                       | Free Tier?                               | Left running                        |
|-----------------------------------------------|------------------------------------------|-------------------------------------|
| NAT Gateway                                   | No                                       | ≈ \$1.1 / day + data (biggest item) |
| Fargate (2 × 0.25 vCPU / 0.5 GB)              | No                                       | ≈ \$0.6 / day                       |
| ALB                                           | 750 h/mo for 12 months on older accounts | ≈ \$0.5 / day otherwise             |
| Public IPv4 (ALB ×2, NAT ×1)                  | partly                                   | ≈ \$0.4 / day                       |
| RDS db.t3.micro, 20 GB                        | 750 h/mo, 12 months (older accounts)     | ≈ \$0.5 / day otherwise             |
| Secrets Manager                               | 30-day trial                             | ≈ \$0.40 / month                    |
| S3, ECR ≤500 MB, CloudFront ≤1 TB, Logs ≤5 GB | Yes                                      | pennies                             |

**TOTAL** ≈ **\$2–3 per day** with everything up. Prices vary by region and change — verify with the AWS Pricing Calculator and set a **\$10 budget alarm** first. Pause cheaply: ECS desired count → 0, delete the NAT Gateway + release its Elastic IP, stop RDS (AWS restarts it after 7 days).

### Teardown order (dependencies first)



Release the NAT's Elastic IP — an unattached EIP is billed. Check that EIPs, NAT gateways, load balancers, RDS, ECS are all empty afterwards.

Figure 13.1 — Delete in this order; AWS refuses to delete things still in use.

## Cheat-sheet

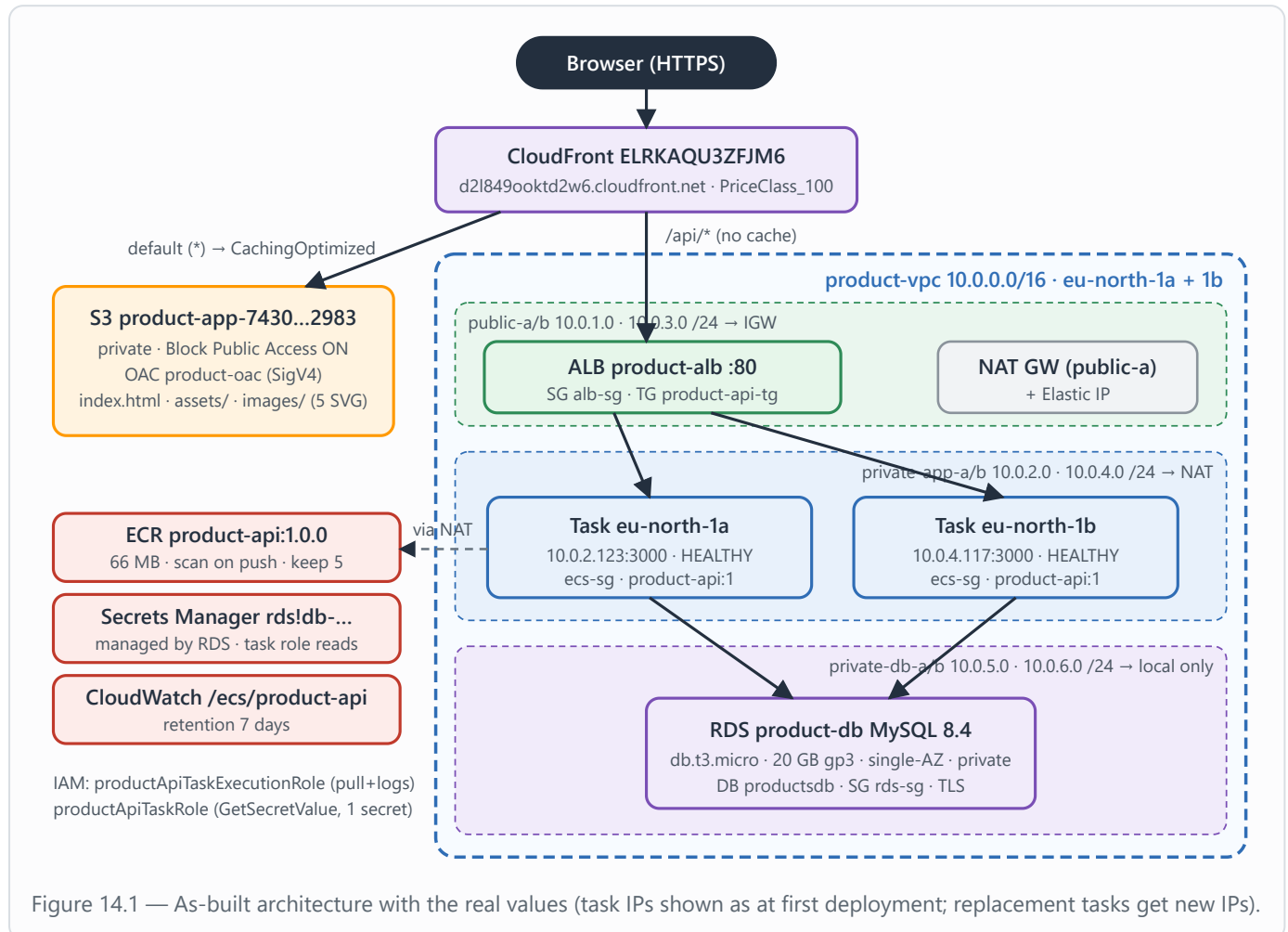
| Service                     | Job here                                                    | Verify with                                                 |
|-----------------------------|-------------------------------------------------------------|-------------------------------------------------------------|
| VPC / subnets / IGW / NAT   | Private network, public front door, outbound-only for tasks | VPC Resource map                                            |
| Security groups             | ALB-SG → ECS-SG:3000 → RDS-SG:3306                          | Reachability Analyzer                                       |
| RDS MySQL                   | Durable shared data (private)                               | <code>describe-db-instances</code> → available, not public  |
| Secrets Manager + IAM roles | DB login handed to the task role only                       | log: <code>db_credentials_loaded</code>                     |
| S3                          | Private image + React storage                               | presigned URL works, public URL 403                         |
| ECR                         | Private image registry                                      | <code>describe-images</code>                                |
| ECS Fargate                 | Runs 2 Node tasks across 2 AZs, rolling deploys             | running = 2, tasks HEALTHY in two AZs                       |
| ALB + target group          | One door; routes only to healthy tasks                      | <code>describe-target-health</code> ; stop a task, stay 200 |
| CloudFront                  | HTTPS + path routing for site, images, API                  | <code>curl -I</code> the three paths                        |
| CloudWatch                  | Logs, metrics, alarms                                       | <code>aws logs tail</code> ; Logs Insights                  |

## Where to go next

- Express all of this as Terraform / CloudFormation / CDK — the console clicks above are the specification.
- ECS service auto scaling (target tracking on CPU 60%); Multi-AZ RDS; one NAT per AZ; VPC endpoints instead of NAT at scale.
- AWS WAF on CloudFront; restrict `alb-sg` to the CloudFront prefix list.
- CI/CD: GitHub Actions with OIDC ( `infra/github-actions-deploy.yml.example` ) — git push → ECR → new task-definition revision → rolling deploy.

## 14 As-built: our live deployment

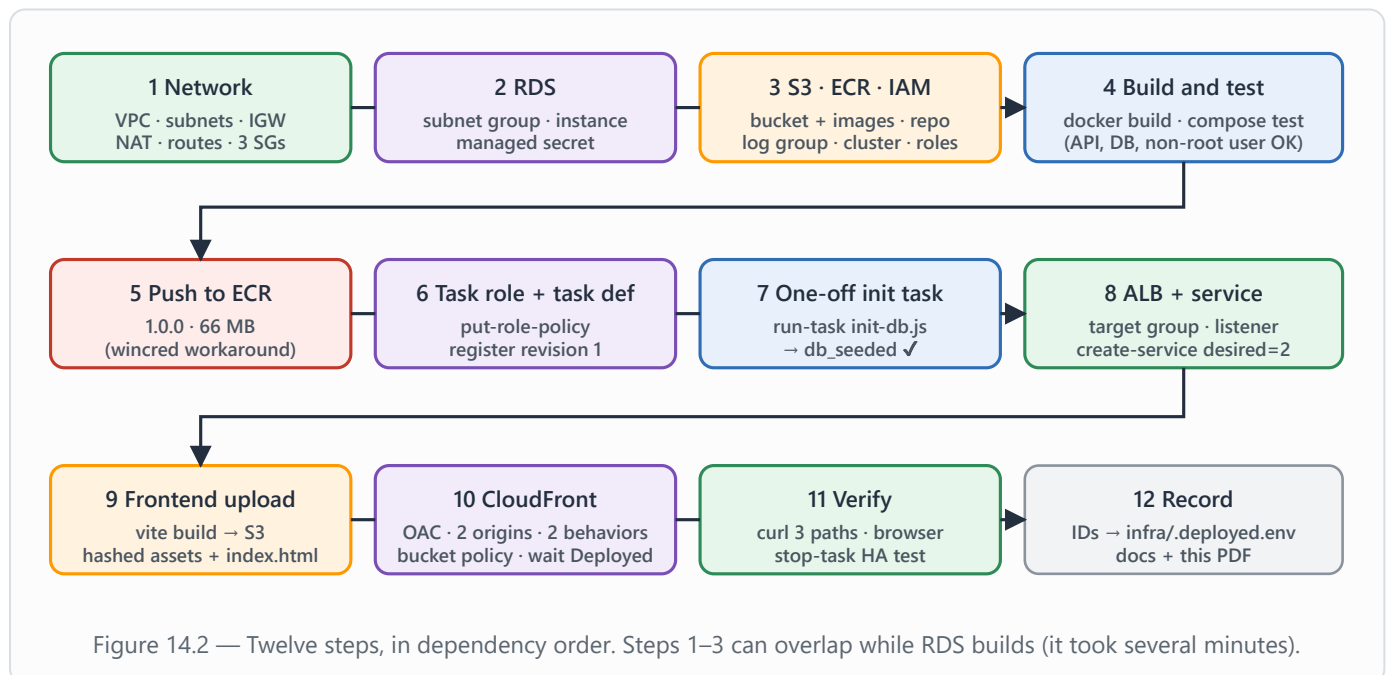
Everything in chapters 4–10 was executed against a real AWS account on **2 October 2026**, in **eu-north-1 (Stockholm)**, using the AWS CLI. This chapter records what exists, how we proved it works, and what went wrong on the way — so another developer can reproduce it or audit it.



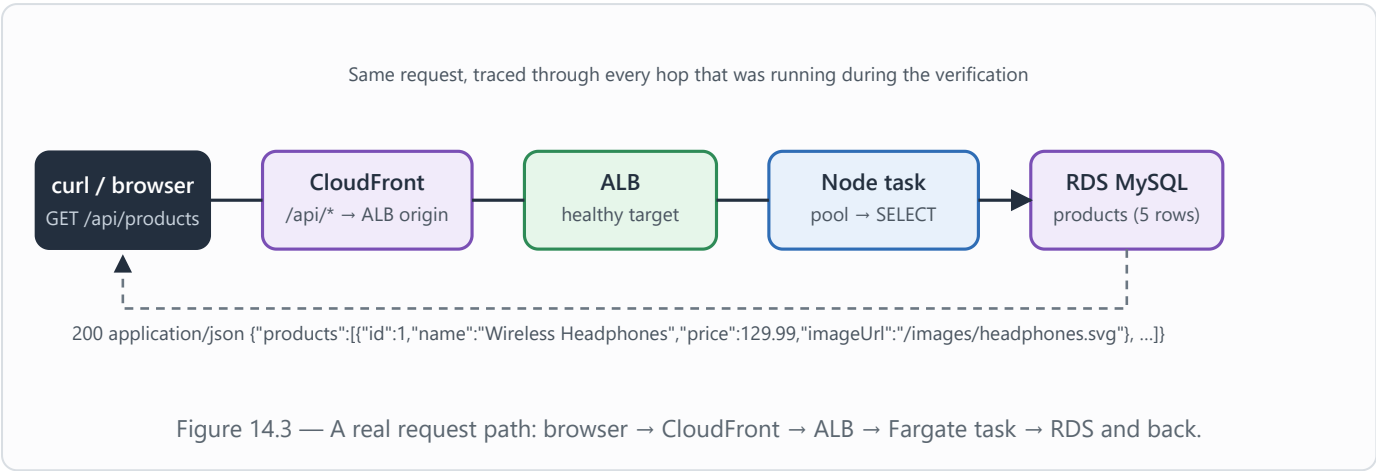
## Resource inventory

| Layer            | Resource                                            | Value                                                                                                                               |
|------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Account / region | role used                                           | 7430...2983 · eu-north-1 · assumed role AccountFullAccessRole , CLI profile default                                                 |
| Network          | VPC, 6 subnets, IGW, NAT + EIP, S3 gateway endpoint | product-vpc 10.0.0.0/16; 2 public, 2 private-app, 2 private-db across eu-north-1a/1b                                                |
| Security groups  | 3 groups                                            | alb-sg (80/443 from internet) → ecs-sg (3000 from alb-sg) → rds-sg (3306 from ecs-sg)                                               |
| Database         | RDS instance + subnet group                         | product-db , MySQL 8.4.9, db.t3.micro , 20 GB gp3, encrypted, backup 1 day, <b>not public</b> , password managed in Secrets Manager |
| Storage / CDN    | S3 + CloudFront + OAC                               | product-app-7430...2983 (private) · distribution ELRKAQU3ZFJM6 · OAC product-oac                                                    |
| Registry         | ECR                                                 | product-api , tag 1.0.0 (66.4 MB), lifecycle: keep last 5                                                                           |
| Compute          | ECS cluster / task def / service                    | product-cluster · product-api:1 (0.25 vCPU, 512 MB) · product-api-service desired 2, circuit breaker + rollback                     |
| Load balancing   | ALB, listener, target group                         | product-alb HTTP:80 → product-api-tg (IP, :3000, /health , deregistration 30 s)                                                     |
| Identity         | 2 IAM roles                                         | execution role (managed policy) · task role (inline: one secret)                                                                    |
| Logs             | log group                                           | /ecs/product-api , 7-day retention                                                                                                  |

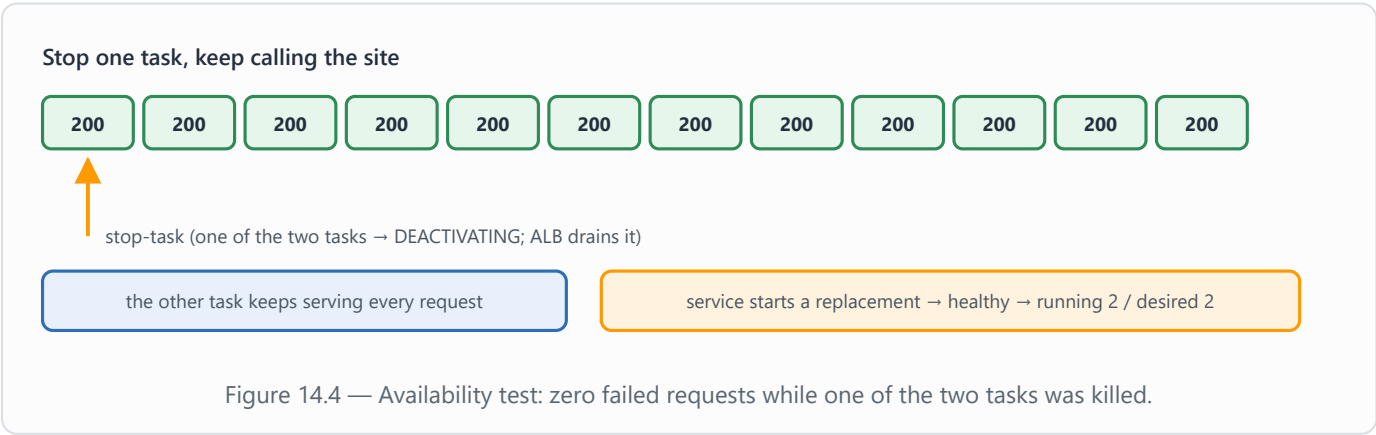
## The order we actually ran things in



Proof that it works



| Check                          | Command / action                                                              | Result                                                                                                 |
|--------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Image runs locally as non-root | <code>docker compose up</code> , <code>exec api whoami</code>                 | <code>node · /health/ready → {"status":"ready", "db":"up"}</code>                                      |
| Secret → DB chain in AWS       | one-off <code>run-task ... init-db.js</code> , <code>aws logs tail</code>     | <code>db_credentials_loaded</code> (secrets-manager) then <code>db_seeded</code>                       |
| Service stable, 2 tasks        | <code>ecs wait services-stable</code>                                         | running 2 / desired 2, tasks in eu-north-1a and 1b                                                     |
| ALB path                       | <code>curl http://ALB/health</code> , <code>/api/products</code>              | <code>{"status":"ok"}</code> , JSON with 5 products                                                    |
| CloudFront paths               | <code>curl -s -o /dev/null -w '%{http_code} %{content_type}'</code>           | <code>/ 200 text/html · /api/products 200 application/json · /images/lamp.svg 200 image/svg+xml</code> |
| Bucket is private              | request the regional S3 URL directly                                          | <b>403 Forbidden ✓</b>                                                                                 |
| Browser rendering              | open the CloudFront URL, check <code>img.naturalWidth</code>                  | 5 product cards; images load (lazy-loaded ones after scroll)                                           |
| High availability              | <code>ecs stop-task</code> on one task; 12 calls through CloudFront every 3 s | <b>12 × 200</b> ; service back to 2/2 after replacement                                                |



## Problems we hit — and the fixes

| # | Symptom                                                                                                                                           | Cause                                                                | Fix                                                                                                                           |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>docker login</code> to ECR → <b>"Error saving credentials ... stub received bad data"</b> ; push then says <i>no basic auth credentials</i> | Windows credential helper cannot store ECR's very long token         | Write the token into a throw-away Docker config ( <code>docker --config &lt;dir&gt; push</code> ), delete the file afterwards |
| 2 | Log group / path arguments starting with <code>/</code> mangled                                                                                   | Git Bash converts <code>/ecs/...</code> to a Windows path            | <code>export MSYS_NO_PATHCONV=1</code>                                                                                        |
| 3 | <code>xargs aws ecs describe-tasks</code> → <i>Unexpected number of separators</i>                                                                | <code>aws.exe</code> prints <code>\r\n</code> and tab-separated ARNs | <code>...   tr '\t' '\n'   tr -d '\r'   xargs</code><br>...                                                                   |
| 4 | <code>create-service-linked-role ecs</code> → <i>name has been taken</i>                                                                          | Role already existed in the account — harmless                       | Ignore; it only needs to exist                                                                                                |
| 5 | Direct S3 check returned <b>307</b> instead of 403                                                                                                | Global S3 endpoint redirects to the bucket's region                  | Test against <code>bucket.s3.&lt;region&gt;.amazonaws.com</code> → 403                                                        |
| 6 | First browser screenshot: grey boxes where images belong                                                                                          | Screenshot taken before lazy-loaded images finished                  | Re-check <code>img.complete</code> / <code>naturalWidth</code> , reload screenshot                                            |
| 7 | Docs said MySQL 8.0                                                                                                                               | RDS default engine is now 8.4.x                                      | App works on both; docs updated                                                                                               |

**WHAT THIS RUN PROVED ABOUT THE DESIGN** Image pull through the NAT, the task role reading the secret, TLS to RDS with the CA bundle baked into the image, the ALB health check that ignores the DB, graceful 2-AZ failover, and path-based routing in CloudFront — all worked on the first attempt with the files in this repository. The only failures were Windows tooling quirks, not AWS design.

## One-page runbook (reproduce or redeploy)

```
# 0  once per terminal (Git Bash)
export MSYS_NO_PATHCONV=1 AWS_REGION=eu-north-1 AWS_DEFAULT_REGION=eu-north-1 && source infra/.deployed.env
# 1  ship a new backend version
docker build -t product-api:1.0.1 backend && docker tag product-api:1.0.1 $ECR_URI:1.0.1
#    push with the throw-away docker config (see chapter 14, problem 1), then:
export IMAGE_URI=$ECR_URI:1.0.1 && bash scripts/render-task-def.sh
aws ecs register-task-definition --cli-input-json file://infra/task-definition.rendered.json
aws ecs update-service --cluster product-cluster --service product-api-service --task-definition product-api
aws ecs wait services-stable --cluster product-cluster --services product-api-service
# 2  ship a new frontend
npm --prefix frontend run build
aws s3 sync frontend/dist s3://$BUCKET --delete --exclude "images/*" --exclude "index.html" --cache-control "public, max-age=31536000, immutable"
aws s3 cp frontend/dist/index.html s3://$BUCKET/index.html --cache-control no-cache
aws cloudfront create-invalidation --distribution-id $CF_ID --paths "/index.html"
# 3  look at it
aws logs tail /ecs/product-api --follow --since 10m
curl -s https://$CF_DOMAIN/api/products | head -c 300
```

## Cost status and teardown for this exact stack

**RUNNING NOW** The stack is live and billing roughly **\$2–3 per day** (NAT Gateway, 2 Fargate tasks, ALB, public IPv4, RDS beyond any free allowance). No budget or CloudWatch/SNS alarms were created. To stop the bill, delete in the order below (all IDs come from `infra/.deployed.env`).

```
source infra/.deployed.env
aws cloudfront get-distribution-config --id $CF_ID # → set Enabled=false, update-distribution with the ETag, wait Deployed, then delete-distribution
aws ecs update-service --cluster product-cluster --service product-api-service --desired-count 0
aws ecs delete-service --cluster product-cluster --service product-api-service --force
aws ecs delete-cluster --cluster product-cluster
aws elbv2 delete-load-balancer --load-balancer-arn $ALB_ARN # wait ~1 min
aws elbv2 delete-target-group --target-group-arn $TG_ARN
aws rds delete-db-instance --db-instance-identifier product-db --skip-final-snapshot --delete-automated-backups
aws rds wait db-instance-deleted --db-instance-identifier product-db
aws rds delete-db-subnet-group --db-subnet-group-name product-db-subnet-group
aws s3 rm s3://$BUCKET --recursive && aws s3api delete-bucket --bucket $BUCKET
aws ecr delete-repository --repository-name product-api --force
aws logs delete-log-group --log-group-name /ecs/product-api
aws cloudfront delete-origin-access-control --id $OAC_ID --if-match <etag>
aws iam delete-role-policy --role-name productApiTaskRole --policy-name read-db-secret && aws iam delete-role --role-name productApiTaskRole
aws iam detach-role-policy --role-name productApiTaskExecutionRole --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
aws iam delete-role --role-name productApiTaskExecutionRole
aws ec2 delete-nat-gateway --nat-gateway-id $NAT_ID # wait until Deleted, then:
aws ec2 release-address --allocation-id $EIP_ID
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids $VPCE_ID
# then delete the 3 security groups (rds-sg, ecs-sg, alb-sg), the 6 subnets, the route tables, detach+delete the IGW, and the VPC
```